

ARCHITECTURE DECISION MEMO

Open Forecast Library

Semantic entity, publisher, database, and blockchain proof architecture

Status	Implemented staging baseline
Version	0.1
Date	12 August 2026
Prepared for	Future Edge Group FZE / iPulse AI
Decision	Adopt Entity -> Target -> Forecast -> Receipt -> Proof -> Evaluation

Recommendation: keep Firestore as the governed operational catalog, use BigQuery only as an analytics mirror, and publish blockchain proofs through a dedicated KMS-controlled Base signer. iPulse AI is the first publisher profile, not the name of the Library.

Executive decision

Adopt an **Entity -> Target -> Forecast -> Receipt -> Proof -> Evaluation** architecture, with explicit publisher and provenance layers around it.

The immediate product should no longer be framed as an "iPulse AI Showcase." It is the **Open Forecast Library**, and iPulse AI is its first publisher profile. A useful public label is:

Forecast Library | Publisher: iPulse AI | Published by Future Edge Group FZE

This preserves three different facts that should not be collapsed:

1. **Future Edge Group FZE** is the accountable legal organization.
2. **iPulse AI** is a product and brand owned and operated by that organization.
3. **iPulse AI Research Team** may be an operating team, but the product itself is not merely a team.

The operational catalog should remain in **Firestore Standard** in [us-central1](#). Firestore is the authoritative system for governed entities, targets, submissions, editorial decisions, receipts, proofs, and public read projections. BigQuery should receive an optional analytics mirror; it should not be the submission or editorial system of record.

For blockchain publication, use a dedicated service wallet controlled by Google Cloud KMS. The web application never receives a private key and visitors do not connect a wallet. Every forecast receipt remains an independent EAS attestation with its own attestation UID, even when multiple attestations are transported in one `multiAttest` transaction.

1. What was confusing and how to fix it

1.1 "iPulse AI Showcase" is too narrow

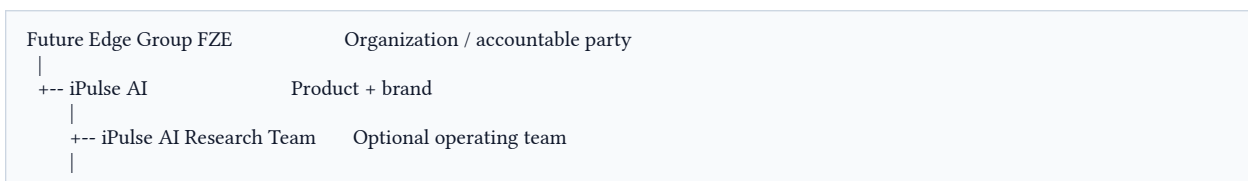
The current phrase makes the Library look like a branded demo rather than a general forecasting infrastructure product. It also mixes the container with one contributor.

Use these concepts instead:

- **Library:** the overall public catalog and verification interface.
- **Collection:** a curated set of forecasts, such as "iPulse AI Market Forecasts - Batch 6."
- **Publisher profile:** the public identity under which a collection is released, such as iPulse AI.
- **Accountable publisher:** the person or organization legally responsible for publication, such as Future Edge Group FZE.
- **Operating team:** an optional team that prepared or managed the release.
- **Submitter:** the authenticated human or service that sent the data to OFL.
- **Forecaster:** the human, AI system, algorithm, ensemble, organization, or hybrid system that generated the forecast.
- **Reviewer:** a human, AI system, algorithm, or organization that reviewed a particular forecast.

These are roles, not synonyms.

1.2 Recommended Future Edge structure



+++ iPulse AI publisher profile Public Library publishing identity

The publisher profile is an application object, not a claim that a brand is a legal person. In public JSON-LD, the Schema.org [publisher](#) should therefore be Future Edge Group FZE, while iPulse AI is represented as the brand/product/source identity. Schema.org defines a brand as a name used by an organization for a product or product group, restricts [publisher](#) to a person or organization, and supports parent/sub-organization relationships and time-qualified organization roles. [1][2][3]

1.3 Do not use "Team" or "Group" as the universal container

"Team" excludes individual publishers and legal organizations. "Group" is informal and conflicts with Future Edge Group's legal name. The universal user-facing term should be **Publisher**. Internally, a publisher profile may point to:

- an accountable organization or person;
- a product or brand;
- an optional operating team;
- one or more authorized submitters.

2. Core semantic model

2.1 The complete chain

Publisher context
 Organization -> Product/Brand -> Team -> Publisher Profile -> Submitter

Forecast knowledge graph
 Domain -> Entity -> Target -> Forecaster -> Forecast -> Receipt -> Proof -> Evaluation

Governance around both
 Candidate -> Editorial Review -> Approved Version -> Public Projection

2.2 Entity, subject, and target

Entity should be the canonical Library term for a real or conceptual thing with persistent identity: a company, listed security, cryptocurrency, person, team, location, event, weather station, crop, disease, policy proposal, or other forecastable object.

Entity is canonical now. The product is still prelaunch, so OFR v0.1 uses [forecast.entity](#); there is no reason to preserve a weaker [forecast.subject](#) contract. In existing iPulse AI code and tables,

`asset` and `prediction subject` are application-role names for this same Entity. Their established column names do not need a global rename, but `asset_id`, `subject_id`, and OFL `entityId` must resolve to the same immutable identity.

Target is not the entity. It is the precisely resolvable variable, question, event, or outcome being forecast. A target may reference multiple entities in named roles.

Examples:

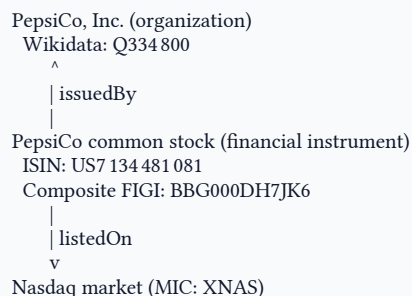
- Entity: PepsiCo, Inc.
- Entity: PepsiCo common stock listed on Nasdaq
- Target: percentage change in adjusted end-of-day close for that security at 20 quarterly steps
- Forecast: Ray Dalio AI persona's predicted sequence of 20 returns

For a non-market example:

- Entity: Abu Dhabi
- Entity: monthly mean air temperature
- Target: mean temperature at Abu Dhabi International Airport during August 2027
- Forecast: a distribution over degrees Celsius

2.3 One identity must not stand in for a related identity

The PepsiCo organization and the PEP-listed security are related but not identical. This matters for the Semantic Web and for ticker changes.



The company entity may safely use `sameAs`: <https://www.wikidata.org/entity/Q334800>. The security must **not** use that company URL as `sameAs`; it should carry security identifiers and an explicit

issuedBy relationship. Schema.org defines sameAs as a page that unambiguously identifies the same thing. [4]

2.4 Stable internal IDs are primary

Every Library entity receives an immutable internal ID. For an entity imported from iPulse AI, OFL reuses the exact deterministic iPulse asset_id as entityId; no second crosswalk ID is invented. For a Library-native entity, OFL generates a deterministic UUIDv5 from a reviewed, immutable identity seed. Environment is never part of that seed, and the same reviewed catalog artifact is promoted to staging and production.

Recommended rules:

- entityId never changes.
- an existing iPulse asset_id is copied unchanged into entityId;
- entityVersionId identifies an immutable approved snapshot.
- a ticker change creates a new identifier assertion with effective dates, not a new company entity;
- a corporate reorganization may create a new entity and a successorOf, mergedInto, or renamedFrom relationship depending on the actual event;
- slugs are routing aliases in a separate collection and may redirect;
- receipt payloads retain the point-in-time entityVersionId used at forecast creation.

3. Alignment with public standards

3.1 Wikidata/Wikibase pattern

Wikidata represents a distinct thing as an item and knowledge as subject-predicate-object statements. Statements can carry qualifiers, references, and ranks. Its pattern is directly useful to OFL: stable entity, typed relationship, effective-time qualifier, source, and editorial rank/status. [5][6]

Use Wikidata as an external semantic crosswalk, not as OFL's primary key or sole authority. These mappings are examples of the distinction that editorial review must preserve:

OFL entity	Wikidata ID	Meaning
PepsiCo, Inc.	Q334800	Organization

OFL entity	Wikidata ID	Meaning
Nvidia	Q182477	Organization
Alphabet Inc.	Q20800404	Organization
Bitcoin	Q131723	Digital cash system and associated currency
SPDR S&P 500 Trust ETF	Q17082716	Exchange-traded fund

Do not put these values into free-form [tags](#). Store each mapping as an identifier assertion with its scheme, original and normalized values, canonical URI, match type, validity interval, source evidence, verification status, and reviewer. A value may appear in a public [sameAs](#) array only after an editor verifies that the Wikidata item and OFL entity are exactly the same thing.

Google Knowledge Graph machine IDs may be captured under the controlled [google_knowledge_graph_mid](#) scheme when an exact match is independently reviewed. They are optional cross-references, not primary keys and not a guarantee of Google indexing. Google's legacy Knowledge Graph Search API is read-only, is not suitable as a production-critical dependency, and directs new users toward Enterprise Knowledge Graph. [26]

For financial instruments, supplement Wikidata with security-specific identifiers. OpenFIGI maps third-party identifiers to FIGIs and returns security type, market sector, ticker, exchange code, composite FIGI, and share-class FIGI. [7] ISO 10383 MICs identify exchanges and trading venues. [8] For legal entities, GLEIF's LEI reference model includes official legal name, registry identity, legal form, formation jurisdiction, creation date, address, and active status. [9]

3.2 Schema.org public pages

Use Schema.org JSON-LD to make public pages machine-readable:

- [Organization](#) and [Person](#) for accountable publishers;
- [Brand](#) and [SoftwareApplication](#) for iPulse AI;
- [Dataset](#) or [CreativeWork](#) for a structured forecast record;
- [PropertyValue](#) for identifiers and measured variables;
- [about](#) for the primary forecast entity;
- [identifier](#), [sameAs](#), [dateCreated](#), [datePublished](#), and [temporalCoverage](#) where appropriate;
- a small OFR vocabulary for concepts Schema.org does not define, including [Forecast](#), [Receipt](#), [Forecaster](#), [Target](#), and [Proof](#).

Schema.org has no native Forecast class. Do not force a misleading type. Use a `Dataset` plus `additionalType` pointing to the OFR vocabulary. [10][11]

3.3 SKOS and provenance

When mappings are not identity claims, use a controlled match type:

- `sameAs` or `skos:exactMatch`: exact identity only;
- `skos:closeMatch`: close concept mapping but not identical;
- explicit relationships such as `issuedBy`, `tracksIndex`, `locatedIn`, or `memberOf` for related entities.

The W3C warns that `exactMatch` is a strong, transitive mapping and distinguishes it from the weaker `closeMatch`. [12] Use the lightweight W3C PROV-O concepts only where they improve interoperability: forecast receipt as an entity, generation as an activity, and forecaster/publisher/reviewer as agents. PROV-O is specifically designed for cross-domain provenance interchange. [13]

4. Proposed data model

4.1 Five distinct object families

A. Parties and operating structure

- `organizations`: legal organizations and organizational publishers.
- `people`: natural persons that submit, publish, forecast, or review.
- `teams`: operating units linked to a parent organization.
- `products`: products, brands, and software applications.
- `memberships`: time-bounded person/team/organization roles.
- `publisher_profiles`: public Library profiles with an accountable party and optional product/team.
- `publisher_authorizations`: who or which service can submit for a publisher.

B. Semantic entity master

- `entities`: stable identity and current approved presentation.
- `entity_versions`: immutable approved snapshots.
- `entity_identifiers`: source-specific identifiers with effective dates and verification evidence.
- `entity_identifier_keys`: unique lookup mapping from normalized external key to `entityId`.
- `entity_relationships`: typed edges between entities, with evidence and effective dates.
- `entity_aliases`: current and historical names, tickers, and route slugs.
- `entity_candidates`: proposed entities or proposed changes awaiting review.
- `editorial_reviews`: review decisions, notes, reviewer, and timestamps.
- `ipulse_adoption_requests`: approved Library entities proposed for selective iPulse onboarding; never a direct write into the iPulse asset dimension.

C. Target registry

- `target_definitions`: reusable measurement/question definitions.
- `target_versions`: immutable target definitions and resolution rules.
- `target_entity_roles`: bindings such as primary security, issuer, venue, location, opponent, or reference index.

D. Forecast and receipt records

- `submissions`: intake state and validation report.
- `forecasts`: queryable forecast index and version references.
- `receipts`: immutable OFR payload and digest.
- `evaluations`: maturity-aware resolution and scoring records.

- `corrections`: append-only correction lineage.

E. Proof and publication operations

- `proof_jobs`: private work queue, one job per receipt/network.
- `proof_runs`: a transport run that may batch multiple independent jobs.
- `public_proofs`: verified attestation UID, transaction hash, chain, schema, and decoded integrity fields.
- `signer_profiles`: public address, KMS key resource reference, chain policy, and status; never private key material.
- `gas_ledger`: estimate, actual gas, fee, exchange-rate reference, and cost center.
- `publisher_runs`: controlled publication audit records.

4.2 Firestore collection boundary

Keep governed/admin collections private and generate denormalized public projections:

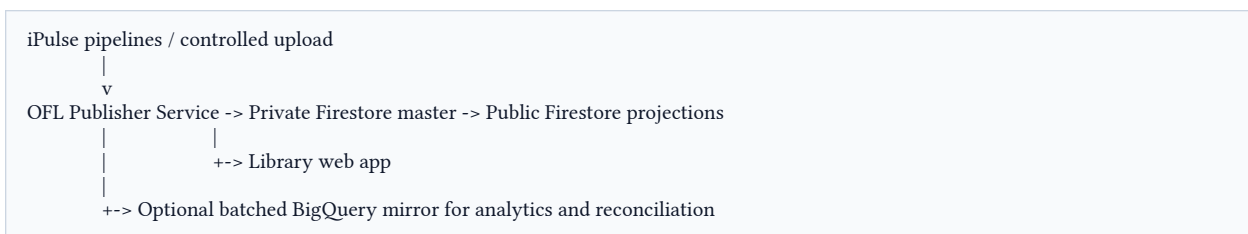
Private / Admin SDK only	Public read-only
organizations	public_publishers
people	public_entities
teams	public_targets
products	public_collections
publisher_profiles	public_forecasters
entities + entity_versions	public_forecasts
entity_candidates + editorial_reviews	public_receipts
target_definitions + target_versions	public_proofs
submissions	
proof_jobs + proof_runs	
signer_profiles + gas_ledger	

The browser receives read access only to `public_*` collections. No browser can write or list private collections. A controlled backend publisher uses the Firebase Admin SDK and IAM to validate and write both governed records and public projections.

4.3 Why Firestore, not BigQuery or SQL, is the operational master

Option	Decision	Reason
Firestore Standard	Use now	Serverless, real-time public reads, IAM/Admin SDK, Security Rules, no idle instance, and a free quota of 1 GiB storage, 50,000 reads/day, and 20,000 writes/day. [14]
BigQuery	Analytics mirror	Excellent for joins, history, scoring, and cross-product analytics; not an editorial CRUD system, public API, or Security Rules replacement. The first 1 TiB of query processing per month is free, making a low-volume mirror inexpensive. [15]
Firebase SQL Connect / Cloud SQL	Do not use now	The service can expose typed APIs, but the underlying Cloud SQL instance starts around \$9.37/month after its limited trial, already above the \$5 budget before hosting or proofs. [16]
External free Postgres	Defer	Adds another vendor, auth surface, migration path, and operational dependency without solving a current scale problem.
Neo4j AuraDB	Optional read replica later	AuraDB Free is positioned for learning and exploration and lacks production controls; AuraDB Professional starts at \$65.70/month for a 1 GB instance, above the OFL budget. [27]
Spanner Graph	Do not use now	Technically strong, but Spanner Graph requires Enterprise or Enterprise Plus and targets substantially larger operational graph workloads. [28]

The recommended low-cost pattern is:



The existing iPulse `dim_fincore_market_assets` table remains the operational master for assets that iPulse actively sources and forecasts. OFL is the semantic master for reviewed identity assertions, external mappings, relationships, and domain-general entities. For iPulse-origin entities there is no artificial crosswalk: `asset_id = subject_id = entityId` at the identity layer.

The first governed snapshot contains all 377 distinct entities with an ACTIVE, FINISHED forecast in iPulse AI scoring batch 6, plus 19 supporting market-venue entities. It is stored as `data/ipulse/scoring-batch-6-entity-catalog.json`, sealed with SHA-256 digest `443f9cb315b4432d9a6be460e7cc82184a847ebba3e818ae0058f1cac92006e7`, and published to `oflapp-staging` as 5,726 Firestore documents. The forecast entities span listed securities, exchange-traded funds, cryptoassets, currency pairs, and commodity spot assets.

4.4 Why a graph-shaped model does not require a graph database yet

Firestore stores the operational knowledge graph as explicit node and edge collections: `entities`, `entity_versions`, `entity_identifiers`, and `entity_relationships`. This preserves graph semantics without paying for a second always-on database. The controlled publisher validates referential integrity and emits JSON-LD plus exportable JSONL or Parquet snapshots.

Add a graph database only when measured use cases require repeated multi-hop traversal, path finding, community detection, graph-based entity resolution, recommendations, or GraphRAG at a scale where bounded Firestore lookups and BigQuery joins are no longer adequate. At that point, feed Neo4j or another graph engine as a disposable read model; do not make it the identity authority. Spanner Graph becomes reasonable only if OFL grows into a mission-critical, high-scale graph workload that can justify Enterprise Spanner.

4.5 When to reconsider Postgres or another operational store

Revisit a relational operational store only when at least one real threshold appears:

- complex multi-entity editorial queries cannot be served by bounded Firestore indexes;
- transactional constraints span many documents and become error-prone;
- third-party publishers require sophisticated row-level tenancy and reporting;
- entity relationships become graph-heavy enough that a search/index layer is justified;
- monthly Firestore read amplification is materially more expensive than a small SQL instance.

Do not migrate because a relational diagram looks cleaner at today's scale.

4.6 Identifier storage and two-way catalog synchronization

Wikidata, Google Knowledge Graph, ISIN, FIGI, MIC, LEI, and future schemes do not belong in the iPulse asset `tags` JSON. Tags are useful for loose filtering; identifiers require uniqueness rules, provenance, temporal validity, and review. Adding one nullable dimension column per external scheme would also create continual Liquibase churn.

Use a normalized iPulse BigQuery table, `xref_fincore_entity_identifiers`, managed by Liquibase. Keep high-value operational columns already present on `dim_fincore_market_assets`, such as ISIN and FIGI, for fast joins, and mirror them into the identifier registry. OFL keeps the richer semantic assertion and review history.

```
iPulse -> OFL
approved dim asset + identifiers
-> deterministic catalog snapshot
-> create or verify entity/version/identifier/edge documents
```

-> public entity projection

OFL -> iPulse (selective)

approved Library entity classified as forecastable_entity + asset

-> ipulse_adoption_request

-> editorial and data-availability review

-> PENDING iPulse onboarding definition

-> normal market-data, fundamental, prediction, and publication gates

-> ACTIVE only after existing iPulse acceptance checks pass

No Library process may insert directly into an ACTIVE iPulse dimension row. The current Python seed files should become bootstrap or migration fixtures rather than the permanent per-asset intake interface. A controlled catalog importer should eventually create governed [PENDING](#) onboarding definitions so approved Library entities can be adopted without editing source code for each asset.

5. Entity master contract

5.1 Example entity document: PepsiCo organization

```
{
  "entityId": "ent_org_7f6d2e17b1b64d899b44b1787d437e0f",
  "entityType": "organization",
  "status": "approved",
  "currentVersionId": "entv_01J5PEPSICO0000000000000001",
  "canonicalName": "PepsiCo, Inc.",
  "description": "American food and beverage company.",
  "schemaOrgTypes": ["Organization", "Corporation"],
  "sameAs": [
    "https://www.wikidata.org/entity/Q334800"
  ],
  "identifiers": [
    {
      "scheme": "wikidata",
      "value": "Q334800",
      "canonicalUri": "https://www.wikidata.org/entity/Q334800",
      "matchType": "same_as",
      "status": "verified"
    }
  ],
  "editorial": {
    "approvedByPartyId": "party_future_edge_editorial",
    "approvedAt": "2026-08-12T00:00:00Z",
    "policyVersion": "ofl-entity-policy-0.1"
  }
}
```

5.2 Example entity document: PEP security

```
{
  "entityId": "equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1",
  "entityType": "listed_security",
  "status": "approved",
  "canonicalName": "PepsiCo common stock",
  "schemaOrgTypes": ["Product"],
}
```

```

"additionalTypes": [
  "https://ipulseai.com/tools/open-forecast-receipt/v0.2/vocab#ListedSecurity"
],
"identifiers": [
  { "scheme": "isin", "value": "US7 134 481 081", "status": "verified" },
  { "scheme": "figi_composite", "value": "BBG000DH7JK6", "status": "verified" },
  { "scheme": "figi_share_class", "value": "BBG001S695T1", "status": "verified" },
  {
    "scheme": "ticker_mic",
    "value": "PEP:YNAS",
    "validFrom": null,
    "validTo": null,
    "status": "verified"
  },
  { "scheme": "ipulse_symbol", "value": "PEP.NASDAQ", "status": "internal" }
],
"sameAs": [],
"relationships": [
  {
    "predicate": "issued_by",
    "objectEntityId": "ent_org_7f6d2e17b1b64d899b44b1787d437e0f"
  },
  {
    "predicate": "listed_on",
    "objectEntityId": "ent_market_xnas"
  }
]
}

```

This intentionally does not claim that the PEP security is `sameAs` the PepsiCo company.

5.3 Example target

```

{
  "targetId": "tgt_market_adjusted_close_step_return_3m_v1",
  "targetVersionId": "tgtv_01J5PEP3M0 000 000 000 000 001",
  "status": "approved",
  "name": "Adjusted end-of-day close step return",
  "targetKind": "numeric_path",
  "domain": "markets",
  "entityRoles": [
    {
      "role": "primary_instrument",
      "entityId": "equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1",
      "entityVersionId": "entv_01J5PEPSEC0 000 000 000 00 001"
    },
    {
      "role": "issuer",
      "entityId": "ent_org_7f6d2e17b1b64d899b44b1787d437e0f",
      "entityVersionId": "entv_01J5PEPSICO0 000 000 000 00 001"
    }
  ],
  "observable": "adjusted_end_of_day_close",
  "transformation": "percentage_change_from_previous_point",
  "outputUnit": "percent",
  "cadence": "P3M",
  "pointCount": 20,
  "resolutionPolicyId": "res_market_adjusted_close_v1"
}

```

The human interface should display `-0.40%` or the underlying decimal `-0.004`, never the internal integer `-40 bps` without a unit label. Storage may use integer basis points for deterministic encoding, but presentation must convert explicitly.

6. Publisher and forecaster contract

6.1 Publisher profile example

```
{
  "publisherProfileId": "pub_ipulse_ai",
  "displayName": "iPulse AI",
  "profileType": "product_brand",
  "accountablePartyId": "org_future_edge_group_fze",
  "productId": "product_ipulse_ai",
  "operatingTeamId": "team_ipulse_ai_research",
  "status": "active",
  "publicDescription": "Open Agentic Investment Research Platform",
  "submissionPolicy": "controlled_internal",
  "editorialPolicyVersion": "off-editorial-0.1"
}
```

6.2 Forecast roles

A forecast record should independently reference:

- `publisherProfileId`: public publishing identity;
- `accountablePublisherPartyId`: organization/person responsible for publication;
- `submittedByPartyId`: authenticated uploader or service;
- `forecasterId` and immutable `forecasterVersionId`;
- zero to ten forecast-specific reviewers;
- model, algorithm, architecture authors, and knowledge-boundary records where relevant.

The Library must not infer that the submitter made the forecast or that the named publisher reviewed it.

7. Entity request and editorial workflow

7.1 Submission against an existing entity

1. Submitter searches the approved entity catalog.

2. Submitter selects the exact entity and target.
3. Publisher service validates that referenced `entityVersionId` and `targetVersionId` are approved.
4. Receipt is generated and sealed.
5. Public index and receipt projection are published.
6. Optional proof job is queued.

7.2 Requesting a new entity

1. Submitter sends an `entity_candidate` with proposed type, name, description, identifiers, relationships, and evidence URLs.
2. The system normalizes candidate keys and searches `entity_identifier_keys` for duplicates.
3. Automated enrichment may propose Wikidata, GLEIF, OpenFIGI, MIC, or other mappings; proposals never auto-approve.
4. Future Edge editorial review chooses **approve**, **merge with existing**, **request changes**, or **reject**.
5. Approval creates an immutable entity version, unique identifier mappings, and a public entity projection.
6. The original forecast submission can then resume against the approved version.

No forecast is publicly released while any required entity or target is unapproved.

7.3 Editorial integrity controls

- two candidate records cannot own the same normalized authoritative identifier;
- mappings carry source, retrieval time, evidence URL/digest, confidence, reviewer, and decision;
- `sameAs` requires exact-identity review;

- entity merges create redirects and lineage; they do not silently replace old receipt references;
- every public entity version is immutable;
- corrections append a new version and retain the original.

This resembles established enterprise master-data patterns. Microsoft's Common Data Model defines extensible semantic entities and stable metadata; its Account model contains a unique object identity and explicit `parentAccountId`. Salesforce account hierarchies similarly require existing parent records and stable Salesforce/external IDs. [17][18]

8. JSON-LD output example

The public PepsiCo forecast page can emit a graph like this:

```
{
  "@context": {
    "@vocab": "https://schema.org/",
    "ofr": "https://ipulseai.com/tools/open-forecast-receipt/v0.2/vocab#",
    "wd": "https://www.wikidata.org/entity/"
  },
  "@graph": [
    {
      "@id": "https://openforecastreceipt.com/entities/ent_org_7f6d2e17b1b64d899b44b1787d437e0f",
      "@type": "Organization",
      "name": "PepsiCo, Inc.",
      "sameAs": "https://www.wikidata.org/entity/Q334800"
    },
    {
      "@id": "https://openforecastreceipt.com/entities/equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1",
      "@type": "Product",
      "additionalType": "ofr:ListedSecurity",
      "name": "PepsiCo common stock",
      "identifier": [
        { "@type": "PropertyValue", "propertyID": "ISIN", "value": "US7134481081" },
        { "@type": "PropertyValue", "propertyID": "FIGI", "value": "BBG000DH7JK6" },
        { "@type": "PropertyValue", "propertyID": "Ticker+MIC", "value": "PEP:XNAS" }
      ],
      "ofr:issuedBy": {
        "@id": "https://openforecastreceipt.com/entities/ent_org_7f6d2e17b1b64d899b44b1787d437e0f"
      }
    },
    {
      "@id": "https://openforecastreceipt.com/receipts/85e82d474841f0497fb7f6fcb0a7fec7ce808de9779ff69eff32a6b6560ea4a5",
      "@type": "Dataset",
      "additionalType": "ofr:Forecast",
      "name": "PepsiCo adjusted close return forecast",
      "about": {
        "@id": "https://openforecastreceipt.com/entities/equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1"
      },
      "identifier": "sha256:85e82d474841f0497fb7f6fcb0a7fec7ce808de9779ff69eff32a6b6560ea4a5",
      "publisher": {
        "@type": "Organization",
        "name": "Future Edge Group FZE",
        "brand": {
          "@type": "Brand",
          "name": "iPulse AI"
        }
      }
    }
  ]
}
```



```

    },
    "dateCreated": "2026-07-05T14:50:00Z",
    "temporalCoverage": "2026-07-05/2031-07-05",
    "variableMeasured": {
      "@type": "PropertyValue",
      "name": "Adjusted end-of-day close step return",
      "unitText": "percent"
    },
    "ofr:forecaster": {
      "@id": "https://openforecastreceipt.com/forecasters/ray-dalio-thinker"
    },
    "ofr:receiptDigest": "85e82d474841f0497fb7f6fcb0a7fec7ce808de9779ff69eff32a6b6560ea4a5"
  }
]
}

```

The example uses an application URL placeholder. Final canonical URLs must use the chosen production domain.

9. Wallet, gas, and proof architecture

9.1 What exists and what does not

The current application has proof-job records, EAS encoding/decoding, verification logic, and Base network integration code. That is not the same as a production wallet service. A secure signer, funding process, transaction policy, and proof-run worker still need to be deployed.

EAS uses a schema registry and attestation contract. An onchain attestation returns its own UID and includes the schema UID, attestor, optional recipient, reference UID, and ABI-encoded data. [19][20]

9.2 Recommended wallet design

Use a dedicated wallet per environment:

- **Staging:** Base Sepolia address, no mainnet assets, funded only with free test ETH from an official faucet.
- **Production:** Base mainnet address controlled by a non-exportable Google Cloud KMS `secp256k1` signing key.

Google Cloud KMS supports `EC_SIGN_SECP256K1_SHA256`; KMS can also sign a 32-byte Keccak digest through the same-size digest interface. Only the proof-run service account receives `cloudkms.cryptoKeyVersions.useToSign`. [21][22]

Firestore stores only:

- wallet address;

- chain ID;
- KMS key resource name;
- allowed EAS contract and schema UIDs;
- funding floor and ceiling;
- per-transaction and per-run cost limits;
- active/suspended status.

It never stores a private key, seed phrase, browser-wallet export, or card/payment credential.

9.3 End-to-end proof run

1. Publisher creates one immutable `proof_job` per receipt and chain.
2. Planner selects compatible jobs by chain, EAS contract, schema UID, and publisher.
3. Planner encodes one EAS attestation per receipt. It may use `multiAttest` for transport efficiency.
4. Preflight estimates both Base fee components, checks signer balance, and converts the estimate to USD using a recorded price source.
5. Policy engine rejects unknown contract calls, unknown schemas, replayed digests, excessive gas, or a run above its USD cap.
6. Cloud Run Job builds an unsigned EIP-1559 transaction and submits its Keccak digest to Cloud KMS.
7. Signer converts the KMS DER signature to Ethereum `r`, `s`, and recovery parity, serializes the transaction, and broadcasts it.
8. Worker waits for confirmation, decodes every emitted EAS UID, and verifies each attestation against the expected receipt digest.
9. Verified `public_proofs` and actual `gas_ledger` rows are written.

10. The iPulse AI batch ledger reads the public proof record and shows "View blockchain proof" for that specific forecast receipt.

Base charges an L2 execution fee and an L1 security/data-publication fee; the L1 component is usually larger and varies with Ethereum conditions. Exact preflight is therefore mandatory. [23]

9.4 Funding policy

Do not send 10 ETH to an application wallet. Fund the production signer with only a small operational float after measuring a complete Base Sepolia run and a mainnet `eth_estimateGas`/L1 fee estimate.

Recommended policy:

- initial float: enough for two approved runs plus a safety buffer, often materially below 0.01 ETH at current Base costs, but determined at execution time;
- balance ceiling: reject top-ups that would put more than the approved float at risk;
- low-balance alert: one future run plus buffer;
- maximum cost: explicit USD cap per transaction and per publisher run;
- manual approval for the first production run and any run above the normal threshold;
- no arbitrary transfers or contract calls from the signer.

Each receipt gets its own attestation UID even when 12 receipts share one transaction. Transaction batching is not receipt aggregation.

9.5 Cost estimate under the \$5 monthly infrastructure budget

Component	Low-volume expectation	Control
Firebase Hosting	Usually \$0 at current traffic	Static build, cache assets
Firestore	Usually \$0 within free quota	Bounded public queries; denormalized projections
Cloud Run Job	Usually \$0 within free tier	Scale to zero; run only for publishing/proofs [24]
Cloud KMS software key	About \$0.06 per active key version/month plus negligible operations	One active key per environment; pricing currently \$0.000082192/hour and \$0.03/10,000 cryptographic operations [25]
BigQuery mirror	Usually \$0 at low volume	Batched load jobs; partitioned queries; max bytes billed
Base Sepolia gas	\$0	Faucet test ETH

Component	Low-volume expectation	Control
Base mainnet gas	Variable, usage-based, outside fixed infrastructure	Preflight and hard USD caps

The fixed architecture can remain below \$5/month at current volume. Mainnet gas cannot honestly be guaranteed as a fixed monthly price because it varies by activity and L1 conditions; it must be recorded and capped as a publication cost.

10. Controlled publisher service

Build the existing publisher script into a small shared library plus two entry points:

1. **CLI:** for Future Edge/iPulse batch publishing and recovery.
2. **Cloud Run Job:** for automated controlled publication and proof processing.

Both use the same validation and idempotency code.

Recommended submission bundle:

```
{
  "bundleVersion": "ofl-publication-bundle/0.2",
  "publisherProfileId": "pub_ipulse_ai",
  "collection": {
    "collectionId": "ipulse-market-batch-6",
    "name": "iPulse AI Market Forecasts - Batch 6"
  },
  "entityBindings": [
    {
      "sourceSubjectId": "equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1",
      "entityId": "equity_ea8243e7-75bd-549b-9ba6-48e03cf982c1",
      "entityVersionId": "entv_01J5PEPSEC0 000 000 000 00 001"
    }
  ],
  "targetVersionId": "tgtv_01J5PEP3M0 000 000 000 000 001",
  "receipts": [
    {
      "receiptPath": "receipts/pepsi-ray.json",
      "proofPolicy": "base_mainnet_selected"
    }
  ]
}
```

Publisher validation must fail closed when:

- a publisher is inactive or unauthorized;
- an entity/target version is missing or unapproved;
- a receipt digest does not recompute;

- a duplicate external identifier points to another entity;
- a public projection would overwrite different immutable data;
- a proof job requests a non-allowlisted chain, contract, schema, or signer.

11. No-regret fields to capture now

These additions are useful immediately and do not force a database migration later. Because the product is prelaunch, the canonical OFR v0.1 schema already uses `forecast.entity`; later semantic additions should still be versioned deliberately rather than silently changing issued receipts.

Publication identity

- `publisherProfileId`
- `publisherDisplayName`
- `accountablePublisherPartyId`
- `productId`
- `operatingTeamId` (optional)
- `submittedByPartyId`
- `publishedByPartyId`

Entity and target identity

- `entityId`
- `entityVersionId`
- `entityType`
- `targetId`

- `targetVersionId`
- `entityRoles[]` with `role`, `entityId`, and `entityVersionId`
- `externalIdentifiers[]` with `scheme`, `value`, `canonicalUri`, `matchType`, `validFrom`, `validTo`, `source`, and `verifiedAt`
- `sameAs[]` only for reviewed exact identity
- `schemaOrgTypes[]`
- `wikidata` and `google_knowledge_graph_mid` as controlled external identifier schemes, never primary keys or free-form tags

Governance

- `editorialStatus`
- `editorialPolicyVersion`
- `approvedByPartyId`
- `approvedAt`
- `supersedesVersionId`
- `correctionOfReceiptId`

Semantic export

- `jsonLdContextVersion`
- `publicCanonicalUrl`
- `publicSlugAliases[]`

Proof operations

- proofPolicy
- proofNetwork
- proofSchemaUid
- proofJobId
- gasCostCenter

12. Immediate implementation sequence

Phase A - low-risk base changes

1. Rename public navigation from "iPulse AI Showcase" to "Forecast Library" or "Browse forecasts."
2. Add publisher/collection filters; label the current material as `Publisher: iPulse AI` and `Published by Future Edge Group FZE`.
3. Add stable publisher, entity, entity-version, target, and target-version references to public projections.
4. Add JSON-LD for the Library landing page, entity pages, and receipt pages.
5. Add private `entity_candidates` and `editorial_reviews`; keep public submission requests directed to support until an admin UI is justified.
6. Extend the controlled publisher to require approved entity/target bindings.

Phase B - governed catalog

1. Maintain the governed snapshot of all 377 current iPulse forecast entities and 19 supporting venues; do not reduce the catalog to the five public showcase assets.
2. Add reviewed organization/instrument relationships and Wikidata, Google Knowledge Graph, ISIN, FIGI, MIC, and iPulse identifier assertions with sources.

3. Generate public entity pages and historical slug redirects.
4. Add optional BigQuery mirror tables for analytics and reconciliation.
5. Add the selective OFL-to-iPulse adoption queue and controlled **PENDING** onboarding importer so new assets do not require per-asset Python seed edits.

Phase C - proof service

1. Deploy staging Cloud Run proof job in `us-central1`.
2. Create dedicated Base Sepolia KMS signer and obtain test ETH.
3. Complete one end-to-end receipt proof, including EAS UID decoding and public verification.
4. Measure gas and define production funding/cap policy.
5. Create separate production KMS signer only after explicit approval.

13. Acceptance criteria

The semantic/entity foundation is ready when:

- iPulse AI appears as a publisher profile, not as the Library itself;
- Future Edge Group is recorded as the accountable organization;
- team, product, publisher, submitter, forecaster, and reviewer are distinct;
- every forecast references approved immutable entity and target versions;
- PepsiCo company, PEP security, Nasdaq venue, Bitcoin, and SPY fund are not conflated;
- ticker/name changes do not break old receipt URLs or identity;
- `sameAs` is used only for exact identity;

- external identifiers are governed assertions rather than free-form tags;
- staging and production use identical entity IDs;
- approved Library-native assets enter iPulse only through a selective **PENDING** adoption gate;
- public pages emit valid JSON-LD;
- no browser can create or modify governed records;
- a controlled publisher can validate and idempotently publish a bundle;
- every onchain receipt has its own EAS UID;
- no private key enters Firestore, source control, the browser, or a local `.env`;
- fixed monthly infrastructure remains below \$5 at current usage, with gas separately measured and capped.

14. Sources

1. Schema.org, Brand: <https://schema.org/Brand>
2. Schema.org, Organization and publisher relationships: <https://schema.org/Organization> and <https://schema.org/publisher>
3. Schema.org, OrganizationRole and parentOrganization: <https://schema.org/OrganizationRole> and <https://schema.org/parentOrganization>
4. Schema.org, `sameAs` and PropertyValue: <https://schema.org/PropertyValue>
5. Wikidata data model: https://www.wikidata.org/wiki/Wikidata:Data_model
6. Wikidata statements and qualifiers: <https://www.wikidata.org/wiki/Help:Statements>
7. OpenFIGI API documentation: <https://www.openfigi.com/api/documentation>
8. ISO 10383 Market Identifier Codes: <https://www.iso20022.org/market-identifier-codes>
9. GLEIF LEI Common Data File reference: <https://www.gleif.org/en/lei-data/access-and-use-lei-data/level-1-data-lei-cdf-3-1-format>
10. Schema.org Dataset: <https://schema.org/Dataset>
11. Schema.org CreativeWork: <https://schema.org/CreativeWork>
12. W3C SKOS reference: <https://www.w3.org/TR/skos-reference/>
13. W3C PROV-O: <https://www.w3.org/TR/prov-o/>
14. Google Cloud Firestore pricing: <https://cloud.google.com/firestore/pricing>
15. Google Cloud BigQuery pricing: <https://cloud.google.com/bigquery/pricing>
16. Firebase SQL Connect pricing: <https://firebase.google.com/docs/sql-connect/pricing>
17. Microsoft Common Data Model overview and Account entity: <https://learn.microsoft.com/en-us/common-data-model/use> and <https://learn.microsoft.com/en-us/common-data-model/schema/core/applicationcommon/account>

18. Salesforce Account Hierarchy: https://help.salesforce.com/s/articleView?id=000384446&language=en_US&type=1
19. Ethereum Attestation Service, how EAS works: <https://docs.attest.org/docs/core--concepts/how-eas-works>
20. Ethereum Attestation Service SDK: <https://docs.attest.org/docs/developer-tools/eas-sdk>
21. Google Cloud KMS algorithms: <https://cloud.google.com/kms/docs/algorithms>
22. Google Cloud KMS signature creation: <https://cloud.google.com/kms/docs/create-validate-signatures>
23. Base network fees: <https://docs.base.org/base-chain/network-information/network-fees>
24. Google Cloud Run pricing: <https://cloud.google.com/run/pricing>
25. Google Cloud KMS pricing: <https://cloud.google.com/kms/pricing>
26. Google Knowledge Graph Search API: <https://developers.google.com/knowledge-graph>
27. Neo4j AuraDB pricing: <https://neo4j.com/pricing/>
28. Spanner Graph overview and editions: <https://cloud.google.com/spanner/docs/graph/overview> and <https://cloud.google.com/spanner/docs/editions-overview>